

Lecture 3: Merge Sort

Ruilin Ma

rm2899@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Merge sort is a divide-and-conquer sorting algorithm. It splits a list into smaller halves, sorts those halves recursively, and then merges them back together in order. Since merging n elements takes linear time and there are about $\log n$ levels of recursion, merge sort runs in $\Theta(n \log n)$ time. This is an improvement over bubble sort, whose worst-case running time is $\Theta(n^2)$. These notes also explain why the recursion terminates, prove that merging and merge sort are correct, and briefly discuss why comparison-based sorting cannot do asymptotically better than $\Omega(n \log n)$ in the worst case.

AI use declaration. I used GPT-6 Astra to research, generating the graph for merge sort, as well as polish the writing and format in the notes. By running the code, I confirmed that the image was generated correctly, and through manual review and verification, I verified that the logic is correct.

1 Motivation

Suppose we have four clinic waiting times: 8, 3, 6, and 2 minutes. After sorting, the values become 2, 3, 6, and 8. Once the data are in order, it becomes easier to identify values such as the middle observations or the longest waiting time. If two patients have the same waiting time, both observations still need to be kept.

One way to sort these values is to repeatedly compare neighboring entries and swap them when they are out of order. This idea works, but for a large list it can require many passes through the data. Merge sort takes a different approach. For example, we can first divide the list into smaller parts, sort them separately, and then combine them. If we have $[3, 8]$ and $[2, 6]$, we compare the first unused value from each list and build the sorted result $[2, 3, 6, 8]$. This combining step is called *merging*.

The main idea in this section is to understand why this splitting-and-merging process works and how much time it takes. Since merge sort is recursive, we also need to see why the recursion eventually stops and why the final output is still a correctly sorted version of the original input. The discussion below uses basic Python lists, simple arithmetic, and summation notation, and the induction steps are written out explicitly.

2 Setup and Notation

Input and output. Before going into the algorithms, we first set up some notation that will be used throughout the notes. Let A be a finite list whose values can be compared with each other. We assume that the comparisons behave normally; for example, if $x \leq y$ and $y \leq z$, then $x \leq z$. As in Python, indexing starts from zero, and the length of a list L is written as $|L|$, or $\text{len}(L)$. We will also use Python-style slicing. The notation $A[lo:hi]$ means that the element at index lo is included, while the element at index hi is not. Therefore, this slice contains $hi - lo$ elements, so we write its size as $n = hi - lo$. For bubble sort, we use n a little differently. In that case, n represents the length of the part of the list that is still being considered, namely the prefix $A[0:n]$.

Definition 1 (Sorting). We say that a list is sorted when its elements are arranged in nondecreasing order. The sorted list should contain exactly the same values as the original input, including any repeated values. By this definition, both an empty list and a list with only one element are already sorted.

Procedure parameters. We assume that all index bounds are integers. For bubble sort, n satisfies $0 \leq n \leq |A|$, while for merge sort we use indices satisfying $0 \leq lo \leq hi \leq |A|$. Table 1 summarizes the meaning of the main arguments used in the algorithms. In the merge sort pseudocode, L and R represent the sorted left and right halves of the list. The variables i and j keep track of the next unused element in each half, and B is the list used to build the merged result.

Procedure	Meaning of the arguments
$\text{SORT}(A, n)$	A : list to modify; n : length of the active prefix.
$\text{BUBBLEUP}(A, j, n)$	j : left index of the next adjacent pair; n : exclusive end of the active prefix. The first call uses $j = 0$.
$\text{MS}(A, lo, hi)$	A : input list; lo : inclusive starting index; hi : exclusive ending index. The corresponding Python function is <code>merge_sort</code> .
$\text{MERGE}(L, R)$	L and R : the two sorted lists to be combined. Inside the procedure, i and j track the next unused elements of L and R , and B stores the merged result. Both indices start at 0 and B starts empty.

Table 1: Parameters used in the pseudocode and Python listings.

Cost and growth. When we talk about the *cost* of an algorithm, we are counting the operations we care about. For example, a key comparison means comparing two values from the input. The total running time also includes other operations such as indexing, assignments, and copying values. In the analysis below, we treat each key comparison and each indexing operation as constant time.

We use asymptotic notation to describe how the running time grows as the input size becomes larger. For nonnegative functions f and g , $f(n) = O(g(n))$ means that, for sufficiently large n , $f(n)$ is bounded above by a constant multiple of $g(n)$. Similarly, $f(n) = \Omega(g(n))$ gives a lower bound. If both the upper and lower bounds have the same growth rate, we write $f(n) = \Theta(g(n))$ (Cormen et al., 2022, pp. 53–57). It is important to remember that O , Ω , and Θ describe growth bounds; by themselves, they do not mean best case or worst case. Unless stated otherwise, all logarithms in these notes use base two.

As a quick comparison of common growth rates, both n^2 and $n \log n$ grow much more slowly than 2^n . Also, even if an algorithm has polynomial running time such as n^p for some fixed $p > 0$, that does not automatically mean that it will be fast enough in practice.

3 Main Development

Bubble sort: one pass and a smaller prefix. On $[3, 2, 1]$, one pass produces $[2, 1, 3]$. A second pass over the first two entries produces $[1, 2, 3]$. Each pass places the active maximum at the right end. The Boolean variables *swapped*, *r*, and *moved* record an exchange in the current pair, the rest of a pass, and the whole pass, respectively. The symbol $\leftarrow$ denotes assignment.

Algorithm 1 Recursive bubble sort and its pass

```

1: function SORT( $A, n$ )
2:   if  $n \leq 1$  then return  $A$ 
3:    $moved \leftarrow$  BUBBLEUP( $A, 0, n$ )
4:   if not  $moved$  then return  $A$ 
5:   return SORT( $A, n - 1$ )
6: end function
7: function BUBBLEUP( $A, j, n$ )
8:   if  $j \geq n - 1$  then return False
9:    $swapped \leftarrow$  False
10:  if  $A[j] > A[j + 1]$  then exchange them;  $swapped \leftarrow$  True
11:   $r \leftarrow$  BUBBLEUP( $A, j + 1, n$ )
12:  return  $swapped$  or  $r$ 
13: end function

```

Listing 1: Python bubble sort. Call `sort(A, len(A))`.

```

1 def sort(A, n):
2   if n <= 1:

```

```

3         return A
4     moved = bubble_up(A, 0, n)
5     if not moved:
6         return A
7     return sort(A, n - 1)
8
9 def bubble_up(A, j, n):
10     if j >= n - 1:
11         return False
12     swapped = False
13     if A[j] > A[j + 1]:
14         A[j], A[j + 1] = A[j + 1], A[j]
15         swapped = True
16     r = bubble_up(A, j + 1, n)
17     return swapped or r

```

The recursive call finishes before `swapped or r` is evaluated. Each pending call retains its own Boolean. The returned value is true if any pair exchanged values. A false result permits early termination because all adjacent pairs were already ordered. These calls modify *A* and return that same list.

Bubble sort's cost. To understand the running time of bubble sort, it helps to first look at a single call to `bubble_up`. Starting from position *j*, the function compares neighboring elements until it reaches the end of the active part of the list. Therefore, for $0 \leq j \leq n - 1$, `bubble_up(A, j, n)` makes exactly $n - j - 1$ key comparisons.

Let $U(j, n)$ represent the running time of this call. Each step does only a constant amount of work before moving to the next position, so we can write

$$U(j, n) = U(j + 1, n) + \Theta(1),$$

with the base case

$$U(n - 1, n) = \Theta(1).$$

Since there are about $n - j$ recursive calls left, this gives

$$U(j, n) = \Theta(n - j).$$

In other words, one individual recursive call only does constant local work, but one complete pass through the active part of the list takes linear time.

Now consider the entire bubble sort. Let $C_B(n)$ be the number of key comparisons in the worst case. A reverse-sorted list is a worst-case example because every pass still needs to be performed.

After one pass over n elements, the largest value has moved to the end, leaving a smaller problem of size $n - 1$. This gives

$$C_B(n) = C_B(n - 1) + (n - 1), \quad C_B(1) = 0.$$

Expanding the recurrence gives

$$C_B(n) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}.$$

So the number of comparisons grows quadratically, and after including the other constant-time operations, the worst-case running time is $\Theta(n^2)$.

The best case is different because this version of bubble sort can stop early. If the input is already sorted, the first pass still makes $n - 1$ comparisons, but no swaps occur, so the algorithm stops after that pass. The best-case running time is therefore $\Theta(n)$.

Merge sort's decomposition. Merge sort sorts two nearly equal slices and merges their sorted results. Let $m := \lfloor (lo + hi)/2 \rfloor$, where the floor rounds down. The slices are $A[lo:m]$ and $A[m:hi]$. Their lengths sum to n , including when n is odd. This is *divide and conquer*: reduce the problem to smaller problems of the same kind, solve them recursively, and combine their results (Cormen et al., 2022, pp. 34–39).

Python implementation. Listing 2 follows the same structure as Algorithm 2. The merge sort itself is recursive, while the `merge` function uses loops to combine the two sorted halves. After one of the two lists is exhausted, the remaining values in the other list are appended one at a time. The function returns a new sorted list, so the original input A is unchanged.

Algorithm 2 Merge sort with an iterative merge

```
1: function MS( $A, lo, hi$ )
2:   if  $hi - lo \leq 1$  then
3:     return  $A[lo:hi]$ 
4:   end if
5:    $m \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
6:    $L \leftarrow MS(A, lo, m)$ 
7:    $R \leftarrow MS(A, m, hi)$ 
8:   return MERGE( $L, R$ )
9: end function

10: function MERGE( $L, R$ )
11:    $B \leftarrow []$ 
12:    $i \leftarrow 0, j \leftarrow 0$ 
13:   while  $i < |L|$  and  $j < |R|$  do
14:     if  $L[i] \leq R[j]$  then
15:       append  $L[i]$  to  $B$ 
16:        $i \leftarrow i + 1$ 
17:     else
18:       append  $R[j]$  to  $B$ 
19:        $j \leftarrow j + 1$ 
20:     end if
21:   end while
22:   while  $i < |L|$  do
23:     append  $L[i]$  to  $B$ 
24:      $i \leftarrow i + 1$ 
25:   end while
26:   while  $j < |R|$  do
27:     append  $R[j]$  to  $B$ 
28:      $j \leftarrow j + 1$ 
29:   end while
30:   return  $B$ 
31: end function
```

Listing 2: Python merge sort. Call `merge_sort(A, 0, len(A))`.

```
1 def merge_sort(A, lo, hi):
2     if hi - lo <= 1:
3         return A[lo:hi]
4
5     m = (lo + hi) // 2
6     left = merge_sort(A, lo, m)
7     right = merge_sort(A, m, hi)
8
9     return merge(left, right)
10
11
```

```

12 def merge(left, right):
13     result = []
14     i = 0
15     j = 0
16
17     while i < len(left) and j < len(right):
18         if left[i] <= right[j]:
19             result.append(left[i])
20             i += 1
21         else:
22             result.append(right[j])
23             j += 1
24
25     while i < len(left):
26         result.append(left[i])
27         i += 1
28
29     while j < len(right):
30         result.append(right[j])
31         j += 1
32
33     return result

```

The cost of merging. Let $s := |L| + |R|$ be the total number of values being merged. If both lists are nonempty, the main comparison loop makes at most $s - 1$ key comparisons. During the whole merge, every value from L and R is appended to B exactly once, so there are s appends in total. Python list appends have constant amortized cost, which means that these s appends take $O(s)$ time altogether. The comparisons and index updates also take linear total time. Therefore, one call to MERGE takes $\Theta(s)$ time.

Termination (the “induction to stop” argument). For the recursive merge sort, the quantity $hi - lo$ tells us the size of the current problem. If $hi - lo \leq 1$, the function stops immediately. Otherwise, the list is divided into two smaller parts. When $n \geq 2$, both parts have sizes between 1 and $n - 1$, so every recursive call works on a strictly smaller problem. Repeating this process eventually reaches a list of size zero or one, which is the base case.

The MERGE procedure is not recursive in this implementation. Instead, it uses three **while** loops. In the first loop, either i or j increases after every comparison. Once one list is exhausted, one of the last two loops appends the remaining elements of the other list, increasing its index each time. Since the indices can never increase past the lengths of the two lists, all three loops must eventually stop.

Why merging is correct. Consider $L = [3, 8]$ and $R = [2, 6]$. The first comparison chooses 2, then the next comparisons choose 3 and 6. At that point, only 8 remains, so it is appended to the result. The final merged list is therefore $[2, 3, 6, 8]$.

The important idea is that, because L and R are already sorted, the smaller of $L[i]$ and $R[j]$ is always the smallest value that has not yet been added to the result.

Lemma 1 (Merge correctness). *Merging two sorted lists returns a sorted list containing exactly the same values as the two input lists.*

Proof. At any point in the main loop, B contains the values that have already been processed, and they appear in sorted order. The next unused values are $L[i]$ and $R[j]$. Since both L and R are sorted, whichever of these two values is smaller must also be the smallest value that has not yet been placed into B . Appending that value therefore keeps B sorted.

After each append, either i or j increases. Eventually one of the two lists is exhausted. The remaining values in the other list are already in sorted order and are appended one at a time by the second or third while loop. Therefore, the final list is sorted and contains every value from L and R , including repeated values. $\square$

Strong induction. We can now use strong induction to show that merge sort works for lists of any size. For a statement $P(n)$ about an input of size n , the strong induction hypothesis assumes that $P(k)$ is true for every smaller size $k < n$. This is useful for merge sort because each recursive call works on a list of roughly half the original size.

Theorem 2 (Merge sort correctness). *For every valid slice $A[lo:hi]$, merge sort returns a sorted list containing exactly the values from that slice.*

Proof. Let $P(n)$ be the claim that merge sort correctly sorts every slice of length n .

Base cases ($n = 0$ and $n = 1$). If the slice contains zero or one element, the function returns $A[lo:hi]$ directly. A list of this size is already sorted and contains exactly the same values as the input slice.

Induction hypothesis. Assume that $P(k)$ is true for every integer $0 \leq k < n$.

Inductive step. For a slice of size $n \geq 2$, merge sort divides it into two parts with sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. Both sizes are smaller than n , so by the induction hypothesis the recursive calls correctly sort both halves. These sorted results are stored in L and R .

The two halves together contain exactly the values from the original slice. By Lemma 1, MERGE combines L and R into one sorted list without losing or adding any values. Therefore, $P(n)$ is also true.

Conclusion. By strong induction, merge sort correctly sorts every nonnegative input size. $\square$

Merge sort's running time. Let $T(n)$ be the running time of merge sort on n values. A list of size zero or one takes constant time. For a larger list, the algorithm makes two recursive calls on lists of roughly half the original size and then performs a linear-time merge. Therefore,

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n).$$

If n is a power of two, this becomes

$$T(n) = 2T(n/2) + \Theta(n)$$

(Cormen et al., 2022, pp. 39–44).

To see where the final running time comes from, consider the recursion level by level. At depth d , there are 2^d subproblems, each containing $n/2^d$ values. Even though there are more subproblems at deeper levels, the total number of values processed across one level is still n . Therefore, the merging work at each level is $\Theta(n)$.

The splitting continues until

$$\frac{n}{2^d} \leq 1.$$

This happens after about $\log_2 n$ levels. Since each level requires $\Theta(n)$ work, the total running time is

$$\Theta(n \log n).$$

Induction for the running time. We can also use induction to check the recurrence for merge sort. For simplicity, suppose n is a power of two and write

$$T(n) = 2T(n/2) + n.$$

We expect the running time to have the form

$$T(n) = n \log_2 n + n.$$

For the base case $n = 1$,

$$T(1) = 1,$$

which agrees with the formula because $\log_2 1 = 0$.

Now assume the formula is true for a problem of size $n/2$:

$$T(n/2) = \frac{n}{2} \log_2 \frac{n}{2} + \frac{n}{2}.$$

Substituting this into the recurrence gives

$$\begin{aligned}
T(n) &= 2T(n/2) + n \\
&= 2 \left(\frac{n}{2} \log_2 \frac{n}{2} + \frac{n}{2} \right) + n \\
&= n(\log_2 n - 1) + 2n \\
&= n \log_2 n + n.
\end{aligned}$$

Thus the recurrence has growth rate $\Theta(n \log n)$. The same asymptotic result holds when n is not a power of two. This implementation always splits the list and processes every value during merging, so both its best case and worst case have running time $\Theta(n \log n)$.

Comparison with bubble sort. Bubble sort and merge sort both compare values and produce the same values in sorted order, but they organize the work differently. Table 2 summarizes the main differences for the implementations used in these notes.

Property	Recursive bubble sort	Python merge sort
Subproblems	One smaller prefix of size $n - 1$	Two parts of about $n/2$
Best / worst time	$\Theta(n) / \Theta(n^2)$	$\Theta(n \log n) / \Theta(n \log n)$
Effect on input	Modifies the original list	Returns a new list

Table 2: Comparison of the bubble sort and merge sort implementations used in these notes for $n \geq 2$.

Can sorting be faster? A *comparison sort* obtains order only through pairwise tests. Its comparison tree has two outcome branches per test and an ordering at each leaf. Let h be the maximum number of tests along a path. At most 2^h leaves must distinguish all $n! := 1 \cdot 2 \cdots n$ possible orders of distinct keys. Correctness requires $2^h \geq n!$, so $h \geq \log_2(n!) = \Omega(n \log n)$ in the worst case (Cormen et al., 2022, pp. 205–208).

4 Worked Example

Let $A = [8, 3, 6, 2, 7, 1, 5, 4]$ be eight waiting times in minutes. The call is `merge_sort(A, 0, 8)`. Figure 1 includes every split and every merged result.

The four pair merges use four comparisons. Each merge into a four-element list uses three. The final merge compares these pairs in order:

$$(2, 1), (2, 4), (3, 4), (6, 4), (6, 5), (6, 7), (8, 7).$$

These comparisons append 1, 2, 3, 4, 5, 6, 7. The remaining 8 is copied without another key

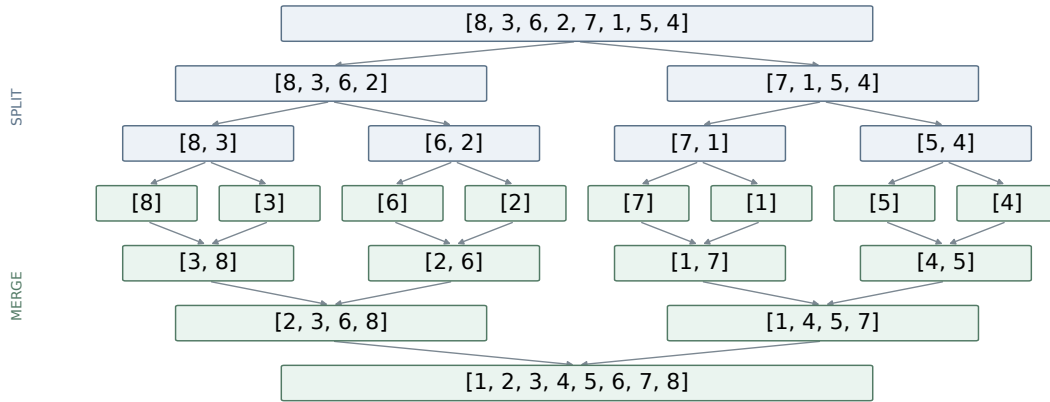


Figure 1: Every split and merge for eight values, generated in Python using Listing 2. Each row preserves all eight values.

comparison. The sorted output is $[1, 2, 3, 4, 5, 6, 7, 8]$. The total is $4 + 3 + 3 + 7 = 17$ comparisons, versus bubble sort's worst-case count of $8 \cdot 7/2 = 28$.

5 Summary

- Bubble sort's linear passes sum to quadratic worst-case time.
- Decreasing sizes ensure termination. Strong induction and merge correctness establish a sorted reordering.
- Merge sort takes $\Theta(n \log n)$ time, matching the worst-case comparison lower bound.

Reference access. Citations use printed page numbers. The [publisher's book page](#) provides access options. Full text may require library access or purchase.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition.